



Name: _____ Cohort/Batch: _____ Date: _____ Score: _____

GOOGLE BIGTABLE PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Databases

TOTAL: 10 QUESTIONS

Q1 What type of database is Google Bigtable and what are its core strengths?

Threat Model

Q6 How do you monitor and tune slow queries in Google Bigtable?

Observability

Q2 How does Google Bigtable guarantee consistency and handle transactions?

Architecture

Q7 What backup and restore strategies does Google Bigtable support?

Lifecycle

Q3 What index types does Google Bigtable support and when do you use each?

Configuration

Q8 How do you handle schema migrations in Google Bigtable?

Compliance

Q4 How do you design a schema or data model in Google Bigtable?

Hardening

Q9 What security features does Google Bigtable provide?

Testing

Q5 How does Google Bigtable scale horizontally?

SSO / IdP

Q10 What are common anti-patterns when using Google Bigtable in production?

Tuning



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Google Bigtable is a relational, document,

Mitigation

Google Bigtable is a relational, document, key-value, graph, or time-series store. Its strengths such as ACID guarantees, horizontal scale, or flexible schema guide the workloads it is best suited for.

A6 Google Bigtable provides a slow query

Observability

Google Bigtable provides a slow query log, explain/explain plan, or profiling tools to surface inefficient queries. Common fixes include adding indexes, rewriting queries, or increasing cache size.

A2 Google Bigtable provides either full ACID

Primitives

Google Bigtable provides either full ACID transactions or eventual consistency depending on its CAP theorem positioning. Transaction support varies from none (simple K/V stores) to serializable isolation (relational DBs).

A7 Google Bigtable supports logical backups

Automation

Google Bigtable supports logical backups (export SQL or BSON), physical backups (filesystem snapshot), and continuous replication to a standby. Point-in-time recovery requires WAL/oplog archiving on top of backups.

A3 Google Bigtable offers B-tree, hash, full-text,

Hardening

Google Bigtable offers B-tree, hash, full-text, spatial, or composite indexes depending on the engine. Choose based on query patterns: B-tree for range queries, hash for equality lookups, full-text for search.

A8 Schema migrations in Google Bigtable are

Governance

Schema migrations in Google Bigtable are managed with versioned migration files applied in order by a migration tool. Migrations should be idempotent and tested in staging before production to avoid downtime.

A4 Schema design in Google Bigtable involves

Gotchas

Schema design in Google Bigtable involves normalizing relations (relational DB) or denormalizing for access patterns (document/key-value). Understanding query needs upfront prevents costly migrations later.

A9 Google Bigtable supports role-based access

Assessment

Google Bigtable supports role-based access control, encrypted connections (TLS), encryption at rest, and audit logging. Always restrict user privileges to the minimum needed and disable anonymous access in production.

A5 Google Bigtable scales via replication (read

Federation

Google Bigtable scales via replication (read replicas) for read-heavy workloads and sharding (partitioning data by key) for write-heavy ones. Some Google Bigtable implementations handle this automatically; others require manual configuration.

A10 Common mistakes include selecting all

Optimization

Common mistakes include selecting all columns instead of needed fields, missing indexes on frequently queried columns, running migrations without backups, and storing large blobs directly in the database instead of object storage.